

Lec 08 Practical Exercise - 1

Lec 08 Practical Exercise - 1

Lecture	08 / 63 + intro
Duration	47:36
Captions	English (auto-generated)
Words	7,292
Video	9xuj1BTwWos
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello all, welcome to the hands-on series on fundamentals of generative AI and large language models theory and practice. In today's session, we'll be demonstrating the use of convolutional neural network in order to predict and classify the images. So let us go through the hands-on. Convolutional neural network makes use of the fundamental operation called as convolution which is multiplication of numbers and then summation of the numbers. So this acts as the base for the convolutional neural networks followed by some pooling operations, use of activation functions and finally the prediction will be done on the images.

[00:42] So let us go through the hands-on. So the first part as usual is importing the necessary libraries. So we are importing the TensorFlow as a TF. TensorFlow is a deep uh deep learning framework which is used to build the neural networks. So this is required in order to have the neural networks. Then from TensorFlow Keras package we are importing the data sets layers as well as models. So data set is required because we may sometimes use uh ready data sets which are available in Kaggle or maybe UCI repository. So in our implementation or today's hands-on we are making use of the SIP 10 data set which is a Canadian institute uh for advanced research data set which uh comprises of 10 different classes of

[01:32] data that is automoto data, vehicle data as well as animals and the birds data. So we are importing such data sets which are available in Google or Kaggle and then importing the necessary layers. So as we all know neural network comprises of three different layers that is the input layer, the dense layer and the output layer. So in order to have the different set of layers we need to import the layers and then to build the convolutional neural network we also need to import the model function. We also import the matplotlib uh lib.pipplot as plt. So matplotlib lib.pipplot Piplot is a data visualization package which which is required for visualizing the data and then we import the numerical Python that is numpy as np. So as the

[02:24] name suggest in order to deal with arrays or numbers we need this numpy package and from skarn which is scikitlearn from the metric sub package we import confusion matrix as well as confusion matrix display. So from scikitlearn metrics we can have uh different metrics in order to measure the performance of the deep learning models or maybe the machine learning models. So out of those metrics we are choosing the confusion matrix or the confusion matrix display which is used to give a comparison between the true label and the predicted label. So we have the actual truth or the ground truth for uh every class that is present in the data set that is for the training data set. Similarly for the test data

[03:06] set. Now whatever model predicts when we compare that predicted output with the true uh output we we need to display that number or we need to display that particular output. So in order to have that uh display we need this confusion matrix display. So this is this is all about importing the necessary libraries. Now coming to loading the data set. So as mentioned earlier we are using the Canadian Institute for Advanced Research data set which comprises of 10 different classes of data mainly the vehicle data the animal and the bird data. So they have 10 classes namely the aeroplanes the automoils birds cats deer dog frog horse ship and the truck. So these are the 10 different classes present in the

[03:56] data set. So what we do in the first line uh in the second code cell is that cifa 10 data set we are loading into various uh variables called as training uh set and the test set. So we have four different variables namely the x train y train as well as x test and the uh y test. So training data comprises of the input data. So that input data uh comes under xrain. Training data will also have the output data which comes under Y train and then the test set will have the input data which can be put to uh called together in X test and it will have the ground truth or the actual uh label or the actual output which can be called as Y test. So this Y test is to be some is something which has to be

[04:44] compared with Y prediction. So that is why earlier we have imported the confusion matrix. So now we are having four different variables in which we have the training set and the test set. Now coming to training data and the test data. So we know that whatever uh data set we have considered this is a set of images. And

when we talk about the grayscale image or the RGB or the color images, they will have values from 0 to 255 because they have 256 different uh levels or 256 different intensity values which vary from 0 to 255. So when we consider this 256 different uh values, it is always difficult for the model uh to train such a data or to learn such uh patterns or to learn those many intensity uh or the pixel values. So in

[05:33] order to make the learner learning easier what we do is we try to normalize the data. So all such values which are varying from the range 0 to 255 we are putting in a range of 0 to 1. So we we have the highest value as 255. That's why we divide the training set as well as test set by 255. The maximum value is 255. So let us go through uh one example. See we we take one RGB or a color image which has three channels the red the green and the blue channel. So here red values can lie between the pixel uh intensity 0 to 255. Similarly this also varies from 0 to 255 as well as blue varies from 0 to 255. So total values are 256 here as well as 256 and 256.

[06:37] The model learning becomes very tedious you know whenever we try to deal with these many numbers. So what we do is since the maximum value is 255 here any intensity that we come across in the image say I take a 4 + 4 image say the pixel value is 100 maybe 50 this is 10 and this is 250 all values lying between 0 to 255 so I divide each of these value 100 by 255 similarly 50 by 255 5. Likewise, if we have this value as 255, then that value is 255 by 255. See, if the we have the minimum value as 0. So that value also to be divided by 0 to 0 by 255. So what happens is in this case the value remains zero. When we divide 255 by the maximum value which is 255, the value is 1. So now we have the normalized pixel values.

[07:47] in the range 0. We have the normalized pixel values in the range 0 to 1. So this is the importance of normalization. Now coming to the different class names as I mentioned earlier there are 10 different classes aeroplane automoto bird cat deer dog frog horse ship and the truck. So these are the 10 different classes present in the sifar 10 data set. So let us now uh print the number of samples present in the training set as well as the test set. So print statement and uh the print statement is training data shape and test data shape and the function used is the shape function on the training set as well as the test set. So once this is run, we are able to see the output training data shape and the test data shape. So there

[08:40] are 50,000 images or 50,000 samples in the training set and 10,000 samples in the test set. And when we look into the shape, what happens is we have each image which has 32 pixels as width, 32 pixels as height and there are three different channels that is RG and B. Same thing uh in case of test data shape that is 10,000 samples we have the we have in the test data set 32 pixels in as width 32 pixels as height and three different channels. So this is all about displaying the data shape both the training as well as the tested data. Now let us display some of the sample images. So for displaying or for visualization we need this smart plot lab uh library. So from plt we have the figure function. Within the figure

[09:30] function we are making use of the argument called as figure size. Uh let us have a figure of 10 in in width and 5 in in height. So these two numbers represent the width and the height of the display or the figure. So for I for uh I values varying in the range 10. So when we write range of 10 there are 10 different values and the index value varies from 0 to 9. So likewise I'm trying to display 10 different images varying from 0 to 9. Let me make use of the subplot function. Let the subplot function have two rows and five different columns. So two rows and five different columns will help us to accommodate 10 different images. And let the iteration vary from 0 like $0 + 1$ becomes 1 1 then $1 + 1$ becomes two.

[10:20] Likewise we go up to the last image. So $I + 1$ number of times and then uh let me show the training samples. So x train for every iteration that I'm going through or for every index that is varying from 0 to 9. Let the title of this image be the class names that is the corresponding class names or the 10 different class names which we have already seen and uh the class names are available in the y train because y represents the labels or the target names which is given for the x train. So let me uh display those class names for every index 0 to 9 and uh plt do.axis axis off. Axis is off because we need not uh represent anything in the x-axis and the y-axis. We need not give any name for the x-axis and the y-axis.

[11:10] That's for that's why we keep the axis as off. So, let us display this particular figure. So, when we try to display this figure, we see the figure as displayed in two different rows and five different columns. Put together there are 10 different images. So we are able to see uh images of frog truck. Uh there are two uh trucks which are displayed. Then we have a image of a deer automo. One more automo horse ship and the cat.

[11:43] So we have seen some of the sample images. Now coming to the major part that is building the convolutional neural network. So as mentioned earlier convolution becomes the fundamental operation under convolutional neural network. So we try to build this models the sequential models because we stack layers one after the other. So we need this uh happening in the sequential manner. So let us have the first convolutional block. So let me add the layers that is uh layers convolutional layer. The first convolutional layer having the number of filters as 32. Let the kernel size be 3, 3 that is $3 + 3$.

[12:22] Activation function used is RLU activation function. And the input shape as defined earlier $32 + 32 + 3$. And then uh after this convolutional layer, we add the max pooling layer of the filter size $2 + 2$. Now let me come to the second convolutional block. Once again add the layers convolutional layers of 64 uh different filters $3 + 3$ is the uh filter size and then the activation function is relu followed by the max pooling layer once again of the size $2 + 2$. Now coming to the third convolutional block with the 128 number of filters $3 + 3$ is the filter size and the activation function is once again ray loop. So we are having three different convolutional layers one after the

other. So we are always free to use the number of

[13:18] convolutional layers as per uh the requirement as per the data set or as per the size of the data set as per the execution or the computation time that is uh really available or may maybe the memory which is available. So in our case we are using three different convolutional layers back to back. But always we should remember a basic CNN can always have one convolutional layer followed by the max pooling layer with some activation functions and immediately come to the dense layer. So uh that is the flattened layer. So once we have these three different convolutional layers we have the flattened layers which will help in converting uh the matrix into a array.

[13:58] So we can see this flatten function which will convert that two 2D array or a 3D array into 1D array. And then coming to the fully connected layer which is the output layer in our case. So output layer which is obtained by uh calling the dense function. Let us have 128 such uh neurons and the activation function is relu. uh after this dense layer let us use the dropout function to drop the uh functioning of certain neurons or to drop the functioning of certain of some of the hidden neurons.

[14:33] So what we are trying to do is we are trying to reduce the 50% of the neurons in order to generalize the model or in order to have a regularized model so that we can avoid the overfitting condition. And at the final output layer we have the dense uh function. Once again since there are 10 classes present in the data set this number of neurons should be always equal to 10. Say we we have two classes then the number of neurons will be equal to two. So we should always remember that the number of classes will define the number of neurons in the tense layer. And finally we have the activation function which is the softmax activation function. So now let us go through step by step what exactly is happening in each of the

[15:16] layers. What is the importance of this convolution? The importance of max pooling. Similarly, importance of the dense or the flattened layer and the dropouts with an example. So let us consider an image maybe a 5 + 5 image. Instead of taking a very heavy image of 32 + 32 as given in the uh sifar data set uh I'll be taking a 5 + 5 data set. So this is a 5 + 5 uh data or a 5 + 5 image which is taken for simplicity. So as mentioned earlier this is a RGB image having three different channels. So one of the channel could be R the second channel could be green and the third channel could be blue. So let me show the experiment on the red channel. Uh and the values are varying from 0 to 255. So let the first value be 10 then 2

[16:16] 1 0 2. So these values represent the pixel intensity. So this is 5 + 5 image and when it is RGB there are three different channels. Now coming to the first convolution functions. Convolution are those layers which will help in uh getting the uh feature extraction that is an image can consist of various points. It can consist of various shape, textures, maybe edges, lines. So all these are the different features which have to be extracted with the help of these features. What happens is as we go deeper in

the network, as we go deeper in the layers, feature extraction becomes more tedious. The filtering becomes more tedious. Unlike the first few layers, number of filters used are very less. We can uh see it here. The

[17:20] number of filters used in the first convolution layer is only 32. Then coming to the second convolutional block, the number of filters used are 64. When coming to the third convolutional block, the number of filters used are 128. So this is all about feature extraction. With the help of different features or the with with the help of different filters or the mask, we are going to extract the features. So at the first layer, it is a basic information which has been extracted. At the second layer, maybe a little higher and when coming to the third block or the third convolutional block, the filtering will be more tedious. So this helps in extraction of the various features present in the data.

[18:05] So coming to the convolution. So convolution makes use of different kernels. So if I'm talking about extraction of edges initially the extraction of edges edges can be extracted using some of the operators like private operator soel operator or canny detector and so on. So let me take an example of private operator. If I'm going to make use of pict operator and I want to detect the vertical edges then a 3 +3 filter would be having a mask of these this way.

[18:52] If I'm talking about the private operator and the horizontal mask then this is the mask which is to be used. Similarly, if we are going to use the soil operator this will be 1 2 1 and this is vertical in nature. If I'm talking about soil horizontal then this will be min -1 - 2 -1 0 0 0 1 2 and 1. So any of these operators could be used. So basically overlapping this particular mask on the given image. So when I try to overlap this particular mask say for example the vertical private on this I need to superimpose on the given image.

[20:00] So nothing but I have to do the pixel multiplication. So 10 into -1 which is - 10 2 into 0 which is 0 + 1 into 1 which is 1 2 into -1 which is -2 5 into 0 which is 0. 4 into 1 which is 4 + 0 into -1 which is 0. 0 into 0 which is 0. And 4 into 1 which is 4. So I need to add all these values. So - 10 + 1 - 9 - 11 this is 7 and this leads to 11. So this is the value which I get after convolution. So what I need to do is this particular mask I need to superimpose at every edge. This way I superimpose on this area. Next I super impose on the third area by making use of windowing technique or make by making use of a parameter called as stride. So in my case if I'm using a stride value of 1 then immediately I need to superimpose

[21:15] this mask on uh the image such that the overlapped pixels are 2 1 0 54 2 044. Again once again if I'm using a stride value as one then my next window will be comprising of value 102 423 44 43 like this I will superimpose this mask on every pixel that is present on the given data. So what is happening here is we are getting the convoluted output which is nothing but the feature map and we when we look

into the output or the feature map the uh size will be reduced when compared to the initial input. So we see here it is a $5 + 5 + 3$ uh image.

[21:57] So this five will be reduced. So how do we find this reduced value is say output width the width of the output image will be equal to the width of the input image minus the filter size plus $2 * \text{padding}$ divided by the stride + 1. So width of the input image width of the input image is five in our case 5 minus the filter size is $3 + 3$. So the filter width is 3 plus $2 * \text{padding}$. Padding is using of zeros in order to maintain the edges. If I'm trying to have zeros throughout you know in the boundaries this way and do perform the convolution then what happens is I'm trying to preserve the edges of the image. So in case I'm using padding then accordingly how many values of uh uh padding I'll be using how many

[23:02] rows or columns of padding I'm using based on that that padding value will be written here. So in my case whatever example I have taken uh zeros are not used. So therefore 2 into 0 which will be equal to 0. And I as I mentioned earlier the stride value will be equal to $1 + 1$. So $5 - 3$ which is $2 / 1 + 1$ which is equal to 3. So after performing the convolution on the given input image which is of the size $5 + 5 + \text{uh}$ 3 I'll be getting the output feature map of the size $3 + 3 + 3$. So this refers to the width, this refers to height and this refers to the number of channels.

[23:49] Same thing same computation to be performed even in case of height. So what happens in case of height is this five is to be used. This particular five is to be used. So W_{out} will be W_{out} for for height. This is W_{out} for width. So this is height input height minus filter + $2 * \text{padding}$ divided by stride + 1. So even in this case it is $5 - 3 + 0 / 1 + 1$ which happens to be 3. So that's why this is written as 3. So the original input is reduced uh to a final size of $3 + 3 + 3$. So what is the benefit of having this? It is memory efficient as well as the computational time will be very less. So this is the benefit of having the convolution as well as it will help in uh extraction of certain features. So as mentioned earlier here

[24:47] we are using the private operator so it will help in detection of the vertical edges. Suppose I'm using the horizontal operator then it will help in detection of the horizontal edges. Similarly in case of soil or private whichever operator we are using vertical or horizontal based on that we'll be able to preserve or detect those edges. So this is the convolution operation. So now my after the convolution we are left out with $3 + 3 + 3$. So now let us see what is pooling here.

[25:23] So now I am left out with a $3 + 3 + 3$ image. This width is three, height is three and number of channels are three. Here this refers to the channels width and then the height. Say for example uh I after convolution I am uh getting the value this value was anyways 11 we had calculated it after certain calculation we get to know the values are this way - 3 1 1 - 1 after calculation we would get P get getting the value this way then what is to be done is we have this pooling we have this pooling

operation.

[26:10] So what is happening in pooling is we are making use of the maximum pooling. So this pooling operation once again helps in reducing the computation time or it is a memory efficient operation. So the pooling could be sometimes the average pooling or minimum pooling. For simplicity we have used maximum pooling which helps in detecting the features which are very much important when compared to the other. So for pooling also we need to define the filter size.

[26:38] Since it is a 3×3 image, I can take blocks of 2×2 size or 1×1 . So uh let me take a let me take the size as 2×2 . So what is the maximum value here out of this? It is 11502. The maximum value is 11. Now this next stride I have the values 5021. So the highest value is 5. In the next window I have 02 minus 1 and 1. So the maximum value is two here. And then the last window it is 2 1 1 and minus 1. The maximum value is two. So this is the max pooling operation. We get to uh have only the maximum values. So once again the $3 \times 3 \times 3$ image is now converted to 2×2 cross. This is done for every channel. So this three remains the same here. So what is the formula? Suppose I'm talking about the width then width

[27:37] of the pooling operation will be width of the input image minus the filters. There is no necessity of padding here because we are not extracting any edges or we are not doing uh any operation on the edges. Instead we are considering four values at a time or maybe 10 values based on the filter size. So padding is not at all necessary here divided by stride + 1. So in our case the width was 3 minus we have taken 2×2 filter size.

[28:08] So width will be 2 divided by stride was always 1×1 . So this will be equal to 2. Similarly output of the height will be the input height height minus the filter divide by stride + 1. So once again this height is $3 - 2$ by 1×1 this remains as 2. So that's why we get the width as two as well as the height as two and channel remains the same. Now on this if we are performing the relu operation which is activation function there are several activation functions you uh that can be used for simplicity once again based on certain uh advantages we make use of the relu function. So relu the functioning of RLU is in such a way that it is maximum value of 0 comma whatever input we have. So if I compare uh uh this

[29:06] image the pulled image. So 0a 11 I'm left left out with 11 0a 5 maximum value is 5 0a 2 it is 2 0a 2 it is 2. What if we had a negative value? Say for example if the obtained pulled image was 11 - 5 2 and 2 then what happens when we do max of 0 comma - 5 is we are getting the value as zero. So after the activation function it uh we will be able to get some nonlinearity in the data that is even negative values will become positive. So it is a one way of adding nonlinearity to the data. So this is all about the relu.

[29:52] So we have covered the convolution, we have covered the max pooling. So

finally what happens in this flattened layer is so let me uh go back to the same example after relu I have these four values which are to be converted into 1D array. So 11 5 2 and 2. These are now my inputs. type can be calling them as the four different features which are extracted from the image. So x_1 , x_2 , x_3 and then x_4 . So these are the different values to these values when we multiply the associated weights and then the bias we get the final output. Those final output are nothing but the raw scores which will be converted into probabilities by making use of the softmax layer. So let us see even that.

[30:50] So from the example we had the inputs as 11 5 2 and 2 which were x_1 x_2 x_3 and x_4 . These were the inputs. This has to be multiplied by the weights. So the weights the number of weights or the matrix uh weight weight matrix will be based on the number of inputs that we have. So what happens is weight will be equal to w_1 , w_2 , w_3 and w_4 . It is based on the number of inputs that we have and the number of columns depends on how many neurons we want in the hidden layers.

[31:38] Say for example, if I want only two two different uh neurons, then this will be w this will be 1 1 2 2 1 2 2 3 1 3 2 4 1 and 4 2. So if I'm trying to draw the network then at the input side I have x_1 x_2 x_3 and then x_4 in the hidden layer. In the hidden layer we have two neurons. That's why we have two different columns. And then at the output layer we have 10 different neurons because we have we should be having 10 different neurons here because we have 10 different classes.

[32:27] So the relation is the final output Z will be equal to $W \text{transpose}$ we have this W we have to take the transpose of this into the input plus the bias. So in our case since there are two neurons at the in the hidden layers bias will be having two values B_1 and B_2 . So this Z will be giving us raw scores they are not at the probability. So how do we convert this into probability is by using the soft max layer. So soft max layer will take the 10 different values. Suppose say suppose I take an example of 10 different values as say 0.5.15 2.1 I take 10 different values. How do I get the probability? This is no more a probability because when we sum up these values, these values are not equal to 1.

[33:35] So how do I get the probability is $e^{\text{power}.5}$ divided by $e^{\text{power}.5}$ plus $e^{\text{power}.15}$ plus $e^{\text{power}.2}$ so on up to we need to take the exponential of all the 10 different values. So now whatever answer we get that's that is the associated probability. So that is the probability of a class uh for the index zero. Similarly, if I want the probability of the class being one if I'm taking that then that will be $e^{\text{power}.15}$ divided by all the exponential values. So likewise I'll be getting the probability for each of the 10 different classes. So whichever probability is high the model is going to predict that particular class.

[34:24] So this is all about the all about building the CNN model. Now quickly we will go through compilation or config configuring the model as well as training the

model. So in while configuring we need to take some optimizers. So in our case we are taking the adam optimizer and the loss is a sparse categorical cross entropy. the uh loss could be either binary cross entropy or categorical cross entropy or sometimes the sparse categorical cross entropy. When we use binary cross entropy is when we have only two classes. Since we have 10 different classes we are left with the option either for uh using the categorical cross entropy or sparse. But in our case we have used the sparse categorical cross entropy because all

[35:10] the 10 different classes what we have they are mapped to 10 different integers. Maybe auto is mapped to value zero. Truck may be uh mapped to value as one. Bird may be two. So we have 10 different integers. So whenever we come across integers, we need to use the sparse CC as the loss function. Uh now let the metrics that we are going to measure be accuracy. So let us measure both the training as well as the test or the validation accuracy. So that's why the matrix is accuracy. Now coming to training of the model. So model which is fit on the training data X train and Y train let the number of epochs number of times uh uh the model has to go through the data is 10 number of times let the batch size be equal to 64. So what is

[35:57] this batch size is? So the first 64 samples 0 to 63 image. For this the model will do the prediction. It will calculate the accuracy. It will calculate the loss. Also it will calculate the weights. Then it will back propagate the it will back propagate the weights. So weight updation takes place at every 64 samples. Now coming to 64 up to the next uh 64 sample 64 to 127. Similarly prediction happens here. Accuracy will be measured. Loss will also be measured and then the weights also be updated. So this refers to the batch how we are separating or how we are grouping different samples into batches. So likewise we have 64. Let us consider 64 different batches. Now let the validation split. Let us uh divide

[36:51] this training uh data into 80% as training and 20% as the validation set. So that's why the ratio is 0.2 validation uh set as 0.2 and verbose equal to one means let us display the result of every epoch in one line. That's why we have one. If we make it zero, there is no displaying of uh the progress bar. If we make verbose equal to two, then it will be displayed in two different lines. So uh we have 10 different uh different epochs the for 10 different epochs we have the accuracy and the loss which is pertaining to the training one we have accuracy as well as loss which is pertaining to the validation set. So as we progress in our epochs the final uh accuracy what we come across is uh 72%.

[37:40] So validation uh set has got an accuracy of 72%. and training accuracy is 75%. So we see that there is no overfitting condition here. Now coming to uh the plotting of the accuracy as well as the loss. So as mentioned earlier figure function is used for plotting with figure size as argument of uh 12 uh inches in width and 5 in in height. So accuracy let us plot this accuracy. Subplot 1 2a 1. So one row, two columns and in the first position let us have the accuracy plotting. Similarly subplot

one row two columns and in the second location let us plot this loss. So we want to we want to plot the training accuracy as well as validation accuracy. Let us give the label as training and validation. Let

[38:32] the title be model accuracy. Let us give some label both on the x-axis as well as the y-axis. So X label is epoch and Y label is accuracy. Same thing to do with loss. Uh labels are training and validation for the training loss and the validation loss. Uh title is model loss, X label and Y label as epoch and loss. And let us have the legend. So legend is this kind of display. So which color holds good for which one. So training represented in blue color, validation represented in orange color. So this portion refers to the legend. So let us show this particular figure. So we see that the model accuracy increases as the number of epochs increases. So we are able to see a display of epoch versus

[39:19] accuracy. Similarly epoch versus loss. So for 10 different epochs we have the accuracy as well as the loss. We see that the accuracy is increasing both for the training as well as the test set. As seen earlier uh the validation accuracy is slightly lesser that is it is nearly equal to 72% whereas the training accuracy was 75%. Same thing to do with the loss. Loss was slightly higher for the validation set even though it is decreasing for every epoch.

[39:52] Now finally coming to the evaluation on the test set. We also have a test set which comprises of 10,000 different samples. So we have this test loss and test accuracy which has to be evaluated on the test data. Now our test data comprises of two different variables X test and the Y test. So let us display both the testing accuracy and the testing loss. So test accuracy is 71% and the test loss is 82. Coming to the prediction, let us see how the true label and the uh predicted label works really works. So let us see the prediction part. Let the built model predict the X test value. Let it be saved in a variable called as predictions. So now uh let us do such predictions for five different samples.

[40:42] So for I in the range uh five 0 to four is the index totally five different samples. predicted label is a variable which I take and let us show the prediction or the prediction is done for which the probability is very much maximum. So we have already seen how to get the probability. So for each of the 10 different classes or the raw score which is available after this particular transformation from the uh three channel to the 1D array whatever we have seen here we have 10 different probabilities whichever argument gets the highest probability that corresponds to a class a proper class. So let us take the argument of the maximum prediction or the maximum probability and let it be saved as predicted label and we always

[41:33] have this actual label which is uh got from the y test which is the actual uh label which is present for the uh test data set. So let us plot it as a figure. So figure is a function which is used. Let us show that uh figure x test uh figure. Let the title be

prediction and the actual uh there there need not be any axis x-axis or y-axis labeling that's why we need to off it and let us show the figures. So what has happened here this for the zeroth argument so prediction one is cat for the actual data which is cat. So that can be seen here.

[42:14] Similarly, we can see the uh shape of the ship here. Actual one is ship, predicted one is also ship. Similarly, once again, we can see the actual one is ship. The predicted one is also ship. Uh the even the fourth sample, the prediction and the actual are the same. Even for the fifth one, the prediction and the actual are the same. But it need not be same for all the test samples which is present. So now let us display through a confusion matrix how many mclassifications and how many correct classifications have taken place. So uh with the help of confusion matrix we are going to display.

[42:51] So let us uh have the probability which is obtained by the largest uh index or the largest argument or the largest value present. So on the predictions and let those be present on the axis equal to 1. So axis is equal to one refers to columns. So all the predicted labels let it be on the y-axis that is this way. This is the predicted ones on the columns. So by default what happens the true labels comes on the rows. It will be by default coming on axis is equal to zero.

[43:26] So let the true labels we got from the y test which is the flattened layer. Flattened refers to 1D array as mentioned earlier. So let us compute the confusion matrix on the true label as well as the predicted label and let us store it in a variable called as cm and let us display the confusion matrix. So confusion matrix display uh confusion matrix which is stored in a variable called as cm. Let us display all the class names the corresponding class names. So let us have a plot. Let the color map be blue. We can have uh different colors here. Let us now have the hues of different uh colors of blue or different hues of blue and x stick rotation equal to 45 here refers to this kind of rotation. If we try to maintain

[44:16] the zero degree what happens is these labels will overlap. So this is all the uh way of beautifi beautifying the display. So let us have this title as CNN cif confusion matrix. So let us show this confusion matrix. So as mentioned earlier on the rows of the confusion matrix we have the true label. On the columns that is access is equal to one we have the predicted label. So this is a 10 + 10 matrix because we have 10 different classes. Obviously uh number of classes will give us the rows and the columns of a confusion matrix. So this is a 10 + 10 matrix and on the diagonal whatever numbers we see those are the correct classifications done by the model. So true label was aeroplane and model has predicted aeroplane. We have

[45:07] 815 such instances or such samples. True label is automo and predicted label is also automobile. So we have 839 such samples. So all these are the correct classification and whatever non-dagonal elements we have those are the false clas-

sifications done by the model. So if I take this example of 35 what has happened is true label which was present in the data set was automo but the model has falsely predicted as aeroplane. So likewise we have 35 such instances. Now if I take this example 16. So true label was automo but the model has falsely predicted it as frog. So likewise I have 16 such examples. So this is the confusion matrix on a cifart data set.

[45:58] So uh the color map what whatever color map we see here hues of uh blue the lower numbers are indicated using the light blue shade and as we go to the higher number highest integer those are represented using the dark blue color. So with the help of confusion matrix it is very much evident that most of the classifications are right which is very much uh seen on the diagonal and the nondagonal elements here here and there there are few mclassifications that is why we we have got only 72% accuracy.

[46:34] So with the help of this particular experiment we have seen the implementation of convolutional neural network which is on a SIP 10 data set which is Canadian Institute for Advanced Research data set having 10 different classes uh mainly the classes uh comprising of vehicles, animals and birds. We have seen how to build a customized CNN model unlike our basic CNN which has one layer of convolution, one layer of pooling and maybe uh certain activation functions and then the dense layer. But what we have tried to do is we have tried to have three different convolution layers having max pooling after which we had the dense layer. So adding more convolution helps in more feature extraction and the with

[47:18] the help of features which are more important prediction becomes very much uh uh easier. In this hands-on we have seen the implementation of convolutional neural network. Thank you.